

Day 10 Handout: Local AI — Running Your Own LLM

AI Application Development Bootcamp · Local inference, model runners, hardware trade-offs, and offline document Q&A

Local vs Cloud AI

Ollama

Quantization & VRAM

Qwen / DeepSeek

llama.cpp / GGUF

vLLM / LocalAI

Offline RAG

Contents

1. **Why Local AI Matters Now** — privacy, offline use, cost at scale, reproducibility
2. **Local vs Cloud AI: The Real Trade-offs** — trade-off table + the RAG pipeline, mapped
3. **Model Architecture Refresher** — transformers, tokens, KV cache, temperature
4. **Local AI Tool Landscape** — Ollama vs Transformers vs llama.cpp vs vLLM vs LocalAI
5. **Hardware Reality: RAM, VRAM, Quantization** — quantization explained + the VRAM calculator
6. **Choosing a Model** — Qwen 3, DeepSeek-R1, and Llama 3.x compared
7. **Ollama API** — REST endpoints, `/api/chat`, `/api/embed`
8. **Practice Project** — 6 scripts: load → chunk → embed → retrieve → answer → UI
9. **Benchmarking** — latency, grounding, refusal, resource use
10. **Security and Privacy** — where does the prompt actually go?
11. **Lab Deliverable**

PRACTICE CODE FILES

Download **code snippets ZIP file** from Moodle and extract it. This handout shows only the key ideas and important fragments from each file, not the full code. When you are ready to run the practices, go to [Section 8 – Practice Project](#). Run each script in order; each one produces a visible result before you continue. Everything runs completely offline once setup is done.

1. Why Local AI Matters Now

In earlier sessions you used a cloud inference provider such as [Groq](#) to call a hosted LLM from Python. That was a good starting point because it let you focus on prompting, memory, and RAG without worrying about local hardware. Day 10 changes the question: instead of "How do I call an LLM API?", we ask "Can I run the model myself, on my own computer or private server?"

Local AI means the model weights are stored on your own machine or infrastructure and inference happens there. This is not automatically better than cloud AI, but for a growing set of real-world situations, it is the only viable option.

Real-world situations where local inference is not optional

Data privacy and legal compliance

When Samsung engineers used ChatGPT to help debug proprietary chip source code in 2023, the code was uploaded to OpenAI's servers as training data, a breach that led Samsung to ban generative AI tools internally.^[*] This pattern repeats across industries. A hospital cannot send patient records to a cloud LLM without violating HIPAA. A law firm cannot paste client contracts into ChatGPT without breaching attorney-client privilege. A bank cannot let trading strategies leave its network. A government agency operating under GDPR cannot use a US-based cloud provider for sensitive citizen data without careful legal structuring. In all these cases, local inference is not a performance choice, it is a compliance requirement.

Air-gapped and offline environments

Many real deployment environments have no reliable internet access: factory floors, offshore platforms, field research stations, military and government systems, and rural healthcare facilities. An AI assistant that requires a cloud API is useless in these settings. A locally-running model on a laptop or edge server works without any network connection after the initial setup — which is exactly what this session's practice project demonstrates.

Cost at scale

Cloud API pricing is designed for moderate usage. At high volume, processing thousands of documents per day, running continuous analysis pipelines, or serving many internal users, the per-token cost accumulates rapidly. A team running 10 million tokens per day at typical GPT-4o-class pricing (roughly \$2.50 per million input tokens, \$10 per million output tokens — check current provider pricing, as it changes often) would spend somewhere in the region of \$2,000–\$4,000 per month, and that scales linearly: a larger org-wide deployment at 10× the volume easily lands in five figures monthly. The same workload on a local 7B model running on a single GPU server costs electricity. Once the hardware is paid off, marginal inference cost approaches zero.^[11]

Reproducibility and model pinning

Cloud providers silently update their models. Code that worked correctly against GPT-4 in January may produce different outputs in July because the underlying model changed. For scientific research, regulatory submissions, audit trails, or any application where consistent outputs matter, this is a serious problem. A locally-pinned [GGUF](#) model file produces identical output every time, the model does not change unless you explicitly pull a new version.

Latency-sensitive applications

Every cloud API call involves a network round-trip to a data centre, queuing behind other users, and the return journey. For interactive applications that need sub-second response times — real-time code completion, live document annotation, on-device voice assistants — this latency is unacceptable. A small local model (1–3B parameters) can respond in under a second on consumer hardware with no network dependency.

Cloud-first AI

Example: Groq, OpenAI API, Anthropic API, Gemini API. The model runs on provider infrastructure. You send prompts and receive responses through an API.

Good when: you need frontier-quality reasoning, managed infrastructure, rapid prototyping, or the data is not sensitive.

Local / self-hosted AI

Example: Ollama on a laptop, Transformers on a workstation, vLLM on a GPU server, llama.cpp on edge hardware.

Good when: data is sensitive or regulated, internet is unavailable, volume is high, or reproducibility is required.

MAIN IDEA

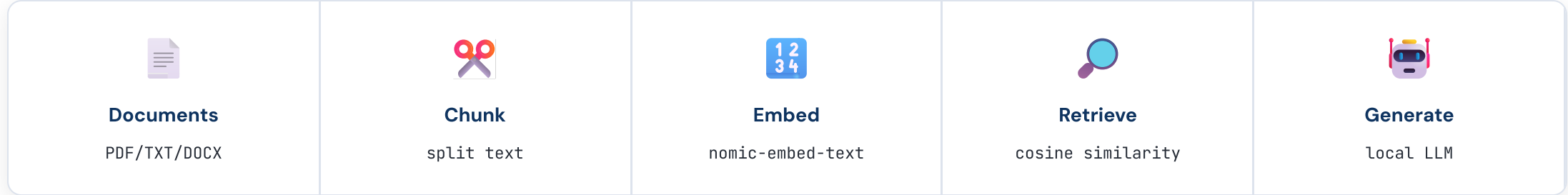
There is no single "best" local AI tool. Ollama is the fastest path to a working local app. Transformers gives full model control. llama.cpp is strongest for CPU/edge inference. vLLM is for production-scale GPU serving. The right choice depends entirely on the project. This handout covers all of them — the lab focuses on Ollama because it gets you running in minutes.

2. Local vs Cloud AI: The Real Trade-offs

A local model is not simply a free version of a cloud model, it changes the application architecture. With cloud inference, your app sends text to a remote provider. With local inference, the model server may be on `localhost`, a private lab machine, or a company GPU server. This can reduce data exposure, but it transfers responsibility to you: model selection, updates, memory management, hardware, and monitoring.

Factor	Cloud AI	Local / Self-hosted AI	Real-world interpretation
Privacy	Prompts leave your system	Prompts stay inside your machine or network	Local is essential for confidential documents, source code, client contracts, medical/legal content, and internal SOPs
Cost	Pay per token/request	Pay for hardware, electricity, maintenance	Cloud is simpler for prototypes. Local wins when usage is repeated and high-volume
Quality	Usually strongest frontier models	Depends on model size and hardware	Small local models are useful but not always comparable to frontier cloud models at complex tasks
Latency	Network + provider queue + model time	No internet round-trip, but hardware may be slower	Local can feel fast for small models; large local models can be slow on weak hardware
Offline use	Usually impossible	Possible after models and packages are installed	Important for classrooms with unreliable internet, air-gapped labs, ships, field sites
Reproducibility	Model versions can change silently	Pinned model version, deterministic	Useful when your application must behave consistently across time
Maintenance	Provider handles model serving	You manage models, runtime, versions, resources	Local gives control but also operational burden

The AI application pattern stays the same. You still design prompts, build memory, perform retrieval, and evaluate answers. What changes is the final generation step. Earlier, the generator was a cloud API call. Today, the generator can be `http://localhost:11434` through Ollama, or any other local endpoint.



3. Model Architecture Refresher: What Runs Locally?

Most modern text-generation LLMs are based on the [Transformer](#) architecture.^[1] You do not need to derive the mathematics, but understanding what happens inside the model during inference explains two practical questions that come up constantly in local AI work: **why does RAM/VRAM matter so much?** and **why does a longer conversation or bigger document slow things down and use more memory?**

The short answers: every model parameter has to live in memory during inference (hence VRAM), and the model needs to keep track of every previous token in the conversation to attend to it (hence the memory cost of long contexts). Click each step below for more detail.



Other generative model families

LLMs are not the only generative AI architecture. A [GAN](#) uses a generator and discriminator competing with each other. A [VAE](#) learns a compressed latent representation. Diffusion models are common in image generation (Stable Diffusion, FLUX). For this session, we focus on Transformer-based LLMs because they are the models you will run through Ollama, llama.cpp, vLLM, or LocalAI.

Key concepts for local inference

Concept	What it means	Why it matters for local AI
<u>Context window</u>	Max tokens processed at once	Determines how much document text you can include in one prompt. Chunking in practice 3 is designed around this limit.
<u>KV cache</u>	Cached attention states for speed	Adds ~20% memory overhead on top of model weights. Rule of thumb: multiply your VRAM estimate by 1.2.
<u>Temperature</u>	Controls output randomness	Use 0.1–0.2 for document Q&A (factual, grounded). Use 0.7+ for creative tasks.
<u>GGUF</u>	Quantized model file format	The format Ollama pulls and stores. When you run `ollama pull qwen2.5:1.5b`, you get a GGUF file.

4. Local AI Tool Landscape: Which Tool for Which Project?

The local AI ecosystem has several layers: model runners, Python frameworks, inference engines, and production serving systems. Comparing them as if they are the same kind of tool causes confusion. The better question is: **what problem are you solving?**

Ollama

Best for local app integration

Simple CLI and REST API for running and calling local models.
[2] Runs on localhost:11434 , no authentication needed for local access. [3] Handles model downloading, GPU/CPU selection, and serving automatically. Under the hood it wraps llama.cpp.

Hugging Face Transformers

Best for control and customisation

Provides model classes, tokenizers, generation settings, and loading utilities such as AutoClasses and from_pretrained(). [4]

Use when: you need fine-tuning, LoRA/PEFT adapters, custom generation, research

llama.cpp / GGUF

Best for lightweight inference

C++ inference engine with minimal setup. Ollama uses it internally. Also usable directly for maximum performance and fine-grained control. [5]

Use when: you need CPU-first deployment, quantised GGUF models, embedded/edge

vLLM

Best for production serving

High-throughput serving with PagedAttention, multiple decoding algorithms, streaming, distributed inference, and an OpenAI-compatible API. [6]

Use when: many users call the model simultaneously and you need batching, GPU efficiency,

Use when: you want a private chatbot, internal document assistant, classroom lab, quick prototype, or a drop-in replacement for a cloud API call — with minimal setup.

experiments, attention/logit inspection, or full control over model internals. Not covered in today's lab — important for when you move beyond using models as-is.

hardware, or the absolute lowest overhead. If you outgrow Ollama's defaults, go here.

and production deployment. Best supported on Linux + NVIDIA GPU, though AMD ROCm and other backends are increasingly supported. Not needed for single-developer or classroom use.

LocalAI

Best for self-hosted OpenAI-compatible API

Free, open-source alternative to OpenAI-style APIs for local or on-prem inference, supporting LLMs, images, audio, and multiple model families.^[7]

Use when: you want a private API server that existing OpenAI-style clients can call with minimal code changes — and you want to keep it fully self-hosted in a company network.

LM Studio / Jan

Best for GUI testing

LM Studio serves local LLMs through a local API server and compatibility endpoints.^[8] Jan is an open-source ChatGPT-style desktop app for local or cloud models.^[9]

Use when: you want no-code local testing, model comparison, or a personal assistant UI before building an app. Both LM Studio and Jan now run on Windows, Mac, and Linux.

🚀 FOR TODAY'S PRACTICE: WE USE OLLAMA

All six practice scripts use Ollama. Not because it is the most powerful option, but because it gives you the shortest path from zero to a working local LLM: one install command, one pull command, and your Python code barely changes from the Groq calls you already know. Once you understand the pattern, swapping to llama.cpp or vLLM for a real project is straightforward.

⚠️ "INSTALLED LOCALLY" DOES NOT MEAN FULLY PRIVATE

A tool being installed on your computer does not automatically mean inference is local. Some desktop AI tools use a local UI but send prompts to a cloud model. For sensitive work, always verify the actual data path: where does the prompt go during inference? Check privacy settings, telemetry, and model provider configuration for any tool you use.

Choose by project requirement

A simplified decision guide based on the tool purposes described in official documentation. Treat it as a starting point, not an absolute rule.

Project goal

Private local app / document assistant



Primary constraint

Simplicity



Recommended starting point: Ollama + local JSON vector store (today's approach)

Shortest path to a working private document assistant. Simple API, no extra services, easy to debug on a laptop.

5. Hardware Reality: RAM, VRAM, and Quantization

The biggest practical barrier to running local LLMs is memory. Every parameter in a model has to live in RAM (CPU) or VRAM (GPU) during inference. A 7B parameter model stored at FP16 — the precision most open-weight models actually ship in — would need around 14 GB. True full precision (FP32) would need close to 28 GB, more than any consumer GPU. This is where [quantization](#) comes in.

What is quantization — and why does it matter?

Quantization compresses each weight into fewer bits, typically 8 or 4. The model becomes much smaller and loads faster, at the cost of a small reduction in numerical precision.^[10] For most text generation tasks, the quality difference between FP16 and Q4 is noticeable but often acceptable, especially at 7B+ parameter scales.

This is the reason you can run a "7B model" on a laptop. Without quantization, a 7B model at FP16 would need ~14 GB of VRAM. The same model at Q4 needs ~4.2 GB.

Conceptual memory shrinkage by precision format

FP32	largest
FP16 / BF16	½ size
INT8 / Q8	¼ size
INT4 / Q4	smallest

Actual memory also includes the KV cache, runtime overhead, and context length. Use the formula below for estimates.

✓

WHEN LLAMA.CPP / GGUF IS THE BETTER REAL-WORLD CHOICE

[llama.cpp](#) is an efficient C/C++ inference engine, it is also what Ollama uses internally. Many local AI tools share the same GGUF model ecosystem. Use llama.cpp directly (without Ollama) when the deployment target is constrained: CPU-only workstation, edge device, embedded system, or a place where Python/ML infrastructure is too heavy. It is not a training framework, it is an inference workhorse. Models from [HuggingFace GGUF models](#) run with it directly.

Memory pressure by model size (Q4 quantized)

Approximate VRAM needed — typical laptop, Q4 quantized

0.5B–1.5B	~0.5–1 GB
3B–4B	~2–2.5 GB
7B–8B	~4.5–5 GB
13B–14B	~8–9 GB
30B+	server-class

The VRAM formula

A useful rule of thumb from [Modal's inference guide](#).^[1]

$$M = (P \times Q / 8) \times 1.2$$
 where M = VRAM (GB), P = parameters (B), Q = bits per weight, 1.2 = KV cache overhead

Parameters (B) Quantization

Q4 — Ollama✓

≈ **4.2 GB VRAM needed** for a 7B model at 4-bit quantization (including KV cache overhead)

Quantization formats

Format	Bits/weight	Quality	Memory vs FP16	Typical use
FP16 / BF16	16	Excellent	Baseline	GPU servers, research, fine-tuning
INT8 / Q8	8	Very good	~50% less	Efficient inference when quality matters
Q4 (GGUF)	4	Good	~75% less	Ollama default – best choice for laptops
Q2 / Q3	2–3	Noticeable loss	~85% less	Only when hardware is very tight

💡 UNIFIED MEMORY VS SEPARATE VRAM — WHAT THIS MEANS FOR YOUR LAPTOP

Most PC laptops have a discrete GPU with its own fixed VRAM pool (typically 4–8 GB), separate from the main system RAM. If a model needs 5 GB and your GPU only has 4 GB, it either fails or falls back to slow CPU inference.

Apple M-series chips and some **AMD APU laptops** use *unified memory*, the CPU and GPU share one memory pool. A 16 GB M2 Mac can give ~12–14 GB to a model. A 16 GB AMD APU laptop works similarly.

On Windows or Linux with a discrete NVIDIA/AMD GPU, Ollama will use your VRAM first, and if the model is too large for VRAM, it offloads layers to RAM automatically (slower, but it still runs). The VRAM formula tells you what will fit at full GPU speed.

6. Choosing a Model: Families, Sizes, and Hardware

There are dozens of open-source model families available on Ollama. Three are highlighted here because they cover the main use cases for this session and are well-supported on most laptop hardware. **Llama** (Meta) is the most widely adopted open-source family — it has the largest fine-tuning ecosystem and is the backbone of many specialised variants. **Qwen 3** (Alibaba) is the best all-rounder for small sizes and multilingual tasks. **DeepSeek-R1** is specialised for reasoning and math. Other families worth knowing exist (Mistral, Phi, Gemma) but are not covered today.

Qwen 3 — Alibaba Cloud

Dense: 0.6B → 32B | MoE: 30B–A3B, 235B–A22B

- Best all-rounder for typical laptop hardware
- 100+ languages and dialects, with strong multilingual instruction following and translation
- **Thinking mode:** add `/think` prefix for chain-of-thought
- 128K context window on larger variants
- MoE models: large model quality, small compute cost
- Top scores on coding benchmarks at each size class

```
ollama run qwen3:4b
```

DeepSeek-R1 — DeepSeek AI

1.5B → 671B (distilled from 671B)

- Specialised for reasoning and math
- Shows full `<think>` reasoning trace before answering
- Rivals GPT-o1 on math benchmarks
- Distilled versions: quality of the 671B model in small packages
- Best for: calculus, proofs, multi-step logic
- Transparent chain-of-thought you can read and verify

```
ollama run deepseek-r1:7b
```

Llama 3.x — Meta AI

1B → 405B

- Most battle-tested open model family — largest community
- Largest fine-tuning and specialised variant ecosystem
- Strong English instruction following
- Great for RAG applications
- Many specialised variants: coding, medical, legal, etc.
- Best for tool calling and agent workflows

```
ollama run llama3.2:3b
```

Hardware recommendation — which size to pull

The table below shows approximate speeds and minimum RAM for each model size. Token-per-second figures are rough estimates — actual speed depends on your GPU, quantization format, and context length. Use this to pick a model that fits your machine, then pull it once before going offline.

Model	Params	Min RAM	Speed (CPU)	Speed (GPU)	Recommendation
qwen3:0.6b	0.6B	4 GB	~12 tok/s	~70 tok/s	Any laptop, fast demos
qwen3:1.7b	1.7B	6 GB	~7 tok/s	~55 tok/s	Lightweight tasks
qwen3:4b 	4B	8 GB	~5 tok/s	~35 tok/s	RECOMMENDED: 8GB RAM+
qwen3:8b	8B	12 GB	~3 tok/s	~25 tok/s	Good quality, 8GB VRAM+
llama3.2:1b	1B	4 GB	~10 tok/s	~60 tok/s	Lightweight, good English instruction
llama3.2:3b	3B	6 GB	~6 tok/s	~40 tok/s	Best for RAG and tool calling at small size
llama3.1:8b	8B	12 GB	~3 tok/s	~24 tok/s	Strong all-round quality, 8GB VRAM+
deepseek-r1:7b	7B	10 GB	~3 tok/s	~28 tok/s	Reasoning tasks
deepseek-r1:14b	14B	16 GB	~1.5 tok/s	~15 tok/s	Best quality (16GB+)

 IF YOUR LAPTOP HAS LESS THAN 8 GB RAM

Use `qwen3:0.6b` , `qwen3:1.7b` , or `llama3.2:1b` . They are smaller but still capable — you will still complete all practice exercises. For the embedding model, `nomic-embed-text` is only 274 MB and works on any machine.

 FOR THIS SESSION

Use `qwen2.5:1.5b` as your primary model for the practices, it is already pulled and works on any laptop. If you have 8+ GB RAM and want better quality answers during the session, try `qwen3:4b` . It has excellent multilingual support (including for Singapore's multilingual context), runs on 8 GB RAM, and has a 32K context window.

7. Ollama API: Using Local Models Programmatically

When Ollama is running, it acts as a local web server at `http://localhost:11434` . Your Python code talks to it by sending HTTP requests, the same protocol your browser uses to load web pages, but from code instead. All practice files use Python's `requests` library to do this. No special Ollama SDK is needed.

HTTP requests come in different types called **methods**. The two you will see here are:

- GET** — ask the server for information without sending any data (like loading a webpage); and
- POST** — send data to the server and get a response back (like submitting a form, or sending a prompt and receiving an answer).

The API shape is close to OpenAI's, so if you later want to switch back to a cloud model, the code change is one line.

Endpoint	Type	What it does	Used in
<code>/api/tags</code>	GET	Ask Ollama which models are installed — used as a health check	practice_01
<code>/api/chat</code>	POST	Send a conversation (system + user messages) and get a reply	practice_01, 05, 06
<code>/api/embed</code>	POST	Send text, receive a vector of numbers representing its meaning	practice_01, 03, 04
<code>/v1/chat/completions</code>	POST	Same as <code>/api/chat</code> but in OpenAI's exact format — swap one line to replace a cloud call	drop-in replacement

Key API pattern: the local LLM call

This is the core replacement for the earlier Groq/cloud call. The full implementation is in `practice_05_local_doc_qa_cli.py` .

Key fragment — chat endpoint

```
response = requests.post(
    "http://localhost:11434/api/chat",
    json={
        "model": "qwen2.5:1.5b",
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        "stream": False,
        "options": {"temperature": 0.2} # low = factual, grounded
    },
    timeout=120
)
answer = response.json()["message"]["content"]
```

✅ OPENAI COMPATIBILITY

Ollama also exposes `/v1/chat/completions` — the same endpoint format as OpenAI's API. If you have existing code using the `openai` Python library, you can point it at Ollama by changing only one line: `base_url="http://localhost:11434/v1"`. The rest of the code stays identical.

8. Practice Project: Offline Multi-file Document Q&A

Six scripts that build a local document Q&A chatbot step by step. Run each in order, each one produces a visible result before you move to the next.

Before you run anything: understand the design choices

Why Qwen 2.5 for the practices?

The practice scripts default to `qwen2.5:1.5b` simply because it is small enough to run on any laptop and was pulled during setup. It is a reasonable choice for a first local model — not the only one, and not necessarily the best for your use case. `llama3.2:1b` or `llama3.2:3b` are equally valid

alternatives and worth trying. To switch, change the model name in the environment variable before running any script:

for example to swap to a different model without editing code

```
# macOS / Linux
export LOCAL_LLM_MODEL=llama3.2:3b

# Windows CMD
set LOCAL_LLM_MODEL=llama3.2:3b
```

The practices use `qwen2.5:1.5b` as the default. DeepSeek-R1 is an alternative if you want to see the model's reasoning chain before the answer. It's slower, but more transparent about how it reached a conclusion.

No LangChain

Previous sessions used LangChain to connect document loaders, embedders, retrievers, and the LLM. These practices deliberately skip it. When you are learning how a local RAG pipeline works, LangChain abstractions hide exactly the steps you need to understand. Here you write each part yourself — file loading, chunking, embedding, retrieval, prompt building — so you can see where things can go wrong and fix them. Once you understand the pipeline, wrapping it back in LangChain is straightforward.

No ChromaDB — plain JSON instead

Earlier RAG sessions used ChromaDB as the vector store. That remains a valid production choice. For this practice the vector store is a plain JSON file. At classroom scale (a few hundred chunks), brute-force cosine similarity takes under 50ms, fast enough that ChromaDB adds no benefit and only adds a dependency. The JSON file is also human-readable: open it and see every chunk and its embedding. When you scale to thousands of documents, swap in ChromaDB, FAISS, or Qdrant.

nomic-embed-text instead of all-MiniLM-L6-v2

`all-MiniLM-L6-v2` through `sentence-transformers` is a perfectly good embedding model. But for today's practices, `nomic-embed-text` through Ollama is simpler: same `/api/embed` call pattern as the chat model, no extra library, fully offline after the initial pull. If you want to switch: `pip install sentence-transformers` and replace the `embed` call with `SentenceTransformer("all-MiniLM-L6-v2").encode(text)`.

⚠️ ONE RULE: NEVER USE A CHAT MODEL FOR EMBEDDINGS

Chat models like Qwen can technically return a vector — but they are 5–10× larger, slower, and not trained for semantic similarity. Always use a dedicated embedding model for any RAG pipeline. `nomic-embed-text` at 274 MB does this job better than a 1.5B chat model.

🔊 DO THIS BEFORE GOING OFFLINE

Run `pip install -r requirements_day10_local_ai.txt`, then pull both models. After that, everything runs locally with no internet.

one-time setup (requires internet)

```
ollama pull qwen2.5:1.5b
ollama pull nomic-embed-text
```

`practice_01_check_ollama.py` ▼

Confirm Ollama is running and both models are installed before touching anything else.

`practice_02_load_many_files.py` ▼

Parse `.txt`, `.md`, `.py`, `.csv`, `.html`, `.pdf`, `.docx` from a `docs/` folder into plain Python strings.

`practice_03_chunk_embed_store.py` ▼

Split text into overlapping chunks → embed each chunk locally → save to a JSON vector store.

`practice_04_retrieve_test.py` ▼

Test retrieval on its own — confirm the right chunks surface for your questions before adding the LLM.

`practice_05_local_doc_qa_cli.py`

First complete answer: retrieve chunks → grounded prompt → local Ollama → answer with citations.

practice_06_final_streamlit_local_doc_chatbot.py

The same pipeline in a local web UI — upload files, tune chunk settings with sliders, ask questions in a browser.

9. Benchmarking: What Should You Measure?

Benchmarking should not be only "which answer sounds better?" A useful local AI comparison includes speed, memory pressure, answer grounding, and refusal behaviour. For this bootcamp, keep the benchmark simple: same question, same retrieved chunks, compare response time and a short human quality score.

Metric	How to measure	Why it matters
Latency	Seconds from request to complete response (<code>time.perf_counter()</code>)	Shows whether the model is usable interactively
Answer grounding	Does it cite and use the provided chunks?	Critical for document Q&A reliability
Refusal behaviour	Ask a question not in the document	Checks whether the model invents unsupported answers
Completeness	Does it include the important details?	Small models may omit key points
Resource usage	Task Manager / Activity Monitor / <code>nvidia-smi</code>	Shows hardware feasibility for production

✓ GOOD BENCHMARK QUESTION SET

Use one factual question clearly answered in the document, one summary question, one comparison question, and one question that is not answered in the documents. This tests speed, grounding, and refusal behaviour, not just surface fluency. Compare at least two model sizes if your hardware allows (e.g. `qwen2.5:1.5b` vs `qwen2.5:3b`).

10. Security and Privacy: Data Path Matters

"Installed locally" does not mean all AI processing is local. Some desktop AI tools use a local UI but send prompts to a cloud model. For sensitive work, the key question is not "Did I install something?" but "**Where does the prompt go during inference?**"

- **Fully local path:** document → local embedding model → local vector store → local LLM endpoint → local answer. This is what practices 01–06 build.
- **Cloud-assisted path:** document or prompt is sent to a hosted model or external API, even if the UI is local.
- **Hybrid path:** retrieval is local, but generation is cloud-based, or vice versa.

For coding assistants and AI IDEs, check privacy settings, telemetry toggles, and whether code snippets are sent to external providers. This is not the same as saying any specific tool is unsafe, the professional framing is that cloud-connected tools may create a data-governance risk depending on your project and jurisdiction (GDPR, HIPAA, internal policy).

✅ What "fully local" means in this project

- Documents never leave your machine
- Embeddings computed locally via Ollama
- Vector store saved to local JSON file
- LLM inference on `localhost:11434`
- Works with no internet after setup

⚠️ Things that would break local privacy

- Switching model to an OpenAI API key
- Using a cloud embedding service
- Storing vectors in a cloud vector DB
- Sending documents to a hosted OCR service
- Logging prompts/responses to a remote server

11. Lab Deliverable

The lab session uses **Lab A: Local AI Developer Assistant** (`day10_lab_local_developer_assistant.ipynb`). This is distinct from the practices: instead of a document Q&A pipeline, you build a coding assistant — explaining code, debugging errors, generating test cases — using a local model via Ollama. The document assistant (practices 01–06) is the guided warmup; the lab is your own build.

Minimum deliverable

- Ollama running locally with a chosen model
- At least 3 developer task prompt templates (explain / debug / test)
- At least 3 example prompts with local model responses
- Benchmark table (response time, model, prompt length)
- Written reflection (150–250 words)

Extension options

- Streamlit or Gradio UI for the assistant
- Model comparison: 0.5B vs 1.5B vs 3B on the same prompts
- Agent-like workflow: classify request type → choose prompt template automatically
- Conversation memory using message history
- Combine with the document pipeline for a code-aware doc assistant

🧠 REFLECTION QUESTIONS

Which tasks did the local model handle well vs poorly?

When would you still prefer a cloud model?

References

- [1] Vaswani et al. "Attention Is All You Need." NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- [2] Ollama API documentation. <https://docs.ollama.com/api/introduction>
- [3] Ollama authentication and local API access. <https://docs.ollama.com/api/authentication>
- [4] Hugging Face Transformers AutoClasses documentation. https://huggingface.co/docs/transformers/en/model_doc/auto
- [5] llama.cpp repository. <https://github.com/ggml-org/llama.cpp>
- [6] vLLM documentation. <https://docs.vllm.ai/en/latest/>

- [7]** LocalAI getting started. https://localai.io/basics/getting_started/index.html
- [8]** LM Studio developer documentation. <https://lmstudio.ai/docs/developer/core/server>
- [9]** Jan official documentation. <https://jan.ai/>
- [10]** Grootendorst, M. "A Visual Guide to Quantization." 2024. <https://www.maartengrootendorst.com/blog/quantization/>
- [11]** Modal. "How much VRAM do I need for LLM inference?" 2024. <https://modal.com/blog/how-much-vram-need-inference>